

Report

Introduction

Dataset: CIFAR-10, containing 60,000 RGB images (3x32x32) with 10 classes

Objective: To build and train a deep neural network for image classification

Network Architecture

Basic Architecture

Intermediate Block:

- The block receives an image x and outputs an image x' computed as weighted sum of multiple convolutional outputs: $x' = a_1 C_1(x) + a_2 C_2(x) + \dots + a_L C_L(x)$
- Convolutional layers: **4 blocks with 2 independent convolution layers each**. All the layers have a **kernel size of 3, padding and stride of 1**. The number of channels is increased by doubling, **64, 128, 256 and, 512** in the 4 successive blocks. All the layers in the same block receive the same input x , and produce outputs $C_1(x)$, $C_2(x)$, ..., $C_L(x)$.
- FC Layer: To compute the weight vector a , the input image x is reduced to a channel-wise average vector m , $m \in \mathbb{R}^c$. This vector m is passed through a fully connected layer to produce the attention weights a , $a \in \mathbb{R}^L$.
- The final output x' is computed by applying a **softmax-normalized** vector a to the stacked convolution outputs, yielding a single fused feature map.

Output Block:

- Receives the final feature map x from the last intermediate block.
- It computes the channel-wise average over spatial dimensions to obtain a vector m . This vector m is then passed through **3 fully connected layers**. The final fully connected layer outputs a logits vector $o \in \mathbb{R}^{10}$ corresponding to the 10 classes.

Modifications from the Basic Architecture

- Channel-wise attention ([SE: Squeeze-and-Excitation](#)) and spatial attention ([CBAM](#)) were experimented with but did not show improvements.
- [Skip connections](#) were also evaluated but did not contribute positively to performance.

Discussion

The original architecture's use of channel-wise averages to weight convolutional outputs effectively functions as an attention mechanism, this could be making the additional attention modules redundant. Skip connections are useful in very deep networks to prevent vanishing gradients. The current network is only 15 layers deep (8 convolutional layers + 7 linear layers). In shallow networks, skip connection don't have much effect, they can even hurt the performance if they reduce the representational power.

The original architecture with 4 intermediate blocks of 2 convolutional layers each and an output block is used for the classification model.

Hyperparameters and Training Techniques

Hyperparameters

- Optimizer: **AdamW**
- Learning Rate (LR): **0.001**
- Weight Decay: **0.01**
- Batch Size: **128**
- Epochs: **50**
- Label Smoothing: **0.1**

Training Techniques

- Preprocessing: **Data normalization**
- Data Augmentations: **Horizontal flips**
- Weight Initialization: **Xavier initialization**
- **Batch Normalization**: used after each block
- **Dropout: 0.3**, used after each block and after the linear layers in the output block
- Activation Function: **LeakyReLU** used after each convolutional layer. **ReLU** used after the linear layers in output block
- Loss Calculation: **Cross-Entropy (CE) Loss**
- Learning Rate Scheduler: **CosineAnnealingLR**

Observations and Discussion

- Data augmentations did not show any significant improvement. Aggressive data augmentation such as *CutMix* seemed to deteriorate performance.
- The **approximate mean and standard deviation of 0.5 for each channel performed better** than the CIFAR mean and std of (0.49139968, 0.48215827, 0.44653124) and (0.24703233, 0.24348505, 0.26158768) respectively. This is likely because 0.5 standardizes the data in a more uniform way across channels.
- Weight Initialization doesn't seem to improve performance, likely due to sufficient training stability from batch normalization and AdamW optimizer.
- **Batch Normalization** and **Dropout significantly improved** the performance and helped avoid overfitting.
- After experimentation with a several different optimizers (SGD, Adam, AdamW) and learning rates, **AdamW** was found to be **optimal** with a LR of **0.001** and a **weight decay of 0.01**.
- ReLU and LeakyReLU show similar overall performance, but LeakyReLU provides a more consistent and gradual improvement in accuracy during training.
- **Label Smoothing** with the CE loss helped **stabilize training** and produced a **smoother loss curve**, reducing overconfidence in predictions.
- Max pooling and average pooling were experimented with, they make the performance worse.

Results

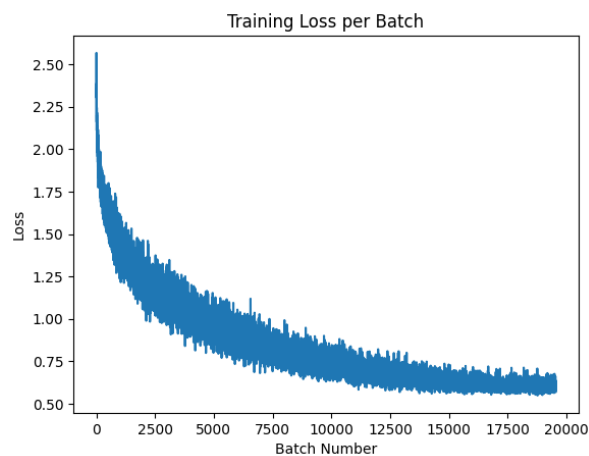
Highest Testing Accuracy: 88.47%

Discussion

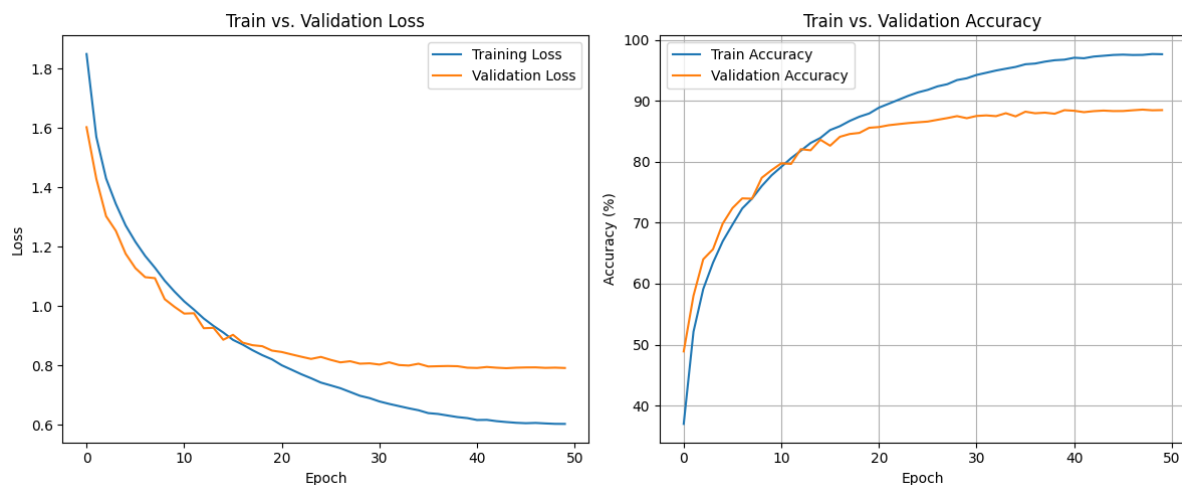
It can be observed in the graph 2a below that the validation loss is initially lower than the training loss. This behavior is likely due to the effect of regularization techniques, such as Batch Normalization and Dropout, which are active only during training and not during evaluation.

The training and validation loss curves show a smooth and consistent decrease, indicating stable training and good generalization. There are no signs of overfitting or divergence.

Both validation loss and accuracy appear to saturate around epoch 35, suggesting that the model reaches its optimal performance around this point. Additional training beyond this epoch results in marginal or no improvement.



Graph 1: Training Loss per Batch



Graphs 2: a) Train vs. Validation Loss per Epoch, b) Train vs. Validation Accuracy per Epoch

Conclusion

A modular CNN was implemented for CIFAR-10 classification using attention-based fusion within intermediate blocks. While SE/spatial attention and skip connections had limited effect, techniques like **AdamW**, **batch normalization**, and **dropout** improved generalization. The model converged well, with validation accuracy stabilizing after ~35 epochs.